Evaluating a RAG Chatbot for JRU CSE Program and Curriculum Inquiries

Luke Joaquin Bernardo
College of Computer Studies and Engineering
Jose Rizal University
Taguig City, Philippines
lukejoaquin.bernardo@my.jru.edu

John Cyruz Patron
College of Computer Studies and Engineering
Jose Rizal University
Mandaluyong City, Philippines
johncyruz.patron@my.jru.edu

Job Russel Enriquez
College of Computer Studies and Engineering
Jose Rizal University
Mandaluyong City, Philippines
jobrussel.enriquez@my.jru.edu

Abstract—We developed and evaluated a retrieval-augmented generation chatbot using the CSE sections of Jose Rizal University's College General Information for school year 2026–2027. We tested five chunking and retrieval configurations with Qwen2.5-1.5B-Instruct and MiniLM embeddings. Earlier matched CPU and T4 GPU experiments used three measured trials per configuration and showed response speedups of 13.063–15.352 times. We selected 800-character chunks, 200-character overlap, top-k of three, and a revised prompt for the final evaluation. That run produced one fresh response to each of 16 questions, including four whose requested details were absent from the source. Six of 12 answerable questions were fully correct, complete, and supported by the supplied passages. Only one of four unanswerable questions was handled correctly, giving 7/16 overall success under the class-specific criteria. Mean response time was 3.580 seconds, excluding setup and warm-up. The results show a practical speed benefit from GPU inference, but course-association errors and invented answers limit the chatbot's reliability.

Index Terms—retrieval-augmented generation, curriculum inquiries, chunking, GPU inference, groundedness, unanswerable questions

I. INTRODUCTION

We built a chatbot for questions about JRU's College of Computer Studies and Engineering (CSE). Our intended users are graduating Grade 12 students exploring its programs. Program descriptions, prerequisites, and curriculum units appear in different handbook sections, so a useful answer may require finding and linking several passages. Our study checks whether the implemented chatbot answers these questions correctly and with enough documentary support.

Retrieval-augmented generation (RAG) combines retrieved knowledge with language generation [1]. In our implementation, a question retrieves handbook passages, and a language model uses them to write an answer. We did not fine-tune the language or embedding model. We evaluated the whole pipeline because an answer can fail when the relevant passage is missing or when the model misreads a passage that was retrieved.

The project relates to SDG 4, particularly access to tertiary education under Target 4.3 [2]. Our contribution is a small technical evaluation of access to program information. We did not measure enrollment decisions, student understanding, or an official SDG indicator. We asked which tested configuration offered a useful balance of speed and answer quality, how matched CPU/GPU workloads differed, and how the selected chatbot handled missing information.

II. SOURCE AND CHATBOT IMPLEMENTATION

A. Source Scope

We used the university's College General Information for school year 2026–2027 [3]. The CSE-only excerpt contains 64 PDF pages, including an image-only cover and 63 readable pages. It covers Computer Engineering, Electronics Engineering, Information Technology, Business Analytics, and the Digital Animation Technology and Game Development specializations. We retained a map between filtered PDF pages, original PDF pages, and printed handbook pages.

Only the approved source documents entered the retrieval index. Questions, reference answers, answerability labels, and scoring files stayed outside it. We preserved source conflicts instead of choosing an unsupported correction. The source does not provide every administrative detail, so tuition amounts, class timetables, and undocumented requirements cannot be confirmed from this knowledge base.

B. Pipeline and Interface

We kept the supplied notebook flow for document loading, chunking, vector storage, model loading, retrieval, and testing. Our generator is Qwen2.5-1.5B-Instruct [4]. pypdf extracts the PDF text, and the recursive character splitter creates overlapping chunks. We embed these chunks with all-MiniLM-L6-v2 [5] and store them in Chroma. At question time, the retriever returns the top-k chunks. The prompt includes their source names and page numbers, followed by the question.

The notebook records the complete prompt and ordered passages, so we can inspect the evidence actually supplied to the model. The implementation includes optional notebook widgets and a terminal chat loop. Each inquiry uses the loaded pipeline independently. We later deployed the same selected pipeline through a browser interface and checked individual requests. We did not evaluate conversation memory or simultaneous users.

C. Hosted Application

We deployed DecisionThree on Hugging Face Spaces with Gradio 6.0.2 and ZeroGPU [9], [10]. The interface uses the JRU logo, blue and gold gradients, and rounded panels. It displays the answer, expandable handbook passages, timing details, and a downloadable JSON response record. A visible notice identifies it as a student research prototype and asks

users to verify requirements in the source. Questions remain independent.

The hosted version keeps Qwen2.5-1.5B-Instruct, MiniLM, float32, the revised prompt, and D settings. Its startup prepares the models, and the first question builds a reusable index. GPU work runs inside the ZeroGPU request function. Downloaded records identify an NVIDIA RTX PRO 6000 Blackwell Server Edition MIG 2g.48gb device. All checked hosted records match the notebook source hash and its 252-chunk fingerprint. Model revisions also match. Hosting changes the hardware and request lifecycle, so matching these controls does not imply identical latency or every generated answer.

D. Model Choice and Fixed Settings

Our proposal named Qwen3-8B [6] as a preliminary model, subject to a hardware pilot. Its four-bit NF4 deployment completed GPU experiments, but two CPU pilots were interrupted before an answer finished. We then used the smaller Qwen2.5-1.5B model to complete matched CPU/GPU experiments. This choice was based on the observed execution difficulty. Different quantization and software settings prevent treating the two deployments as a controlled model-only comparison.

For the main experiments, we kept float32, eager attention, greedy decoding, seed 42, a maximum of 256 new tokens, and two Torch CPU threads. The recorded checkpoint revisions begin 989aa7980e4c for Qwen2.5 and 1110a243fdf4 for MiniLM. Full revision identifiers are in the saved controls. The embedding and language models ran on the selected device; Chroma search remained on CPU. We used no external language-model API for these runs.

Recorded software versions were Torch 2.11.0, Transformers 5.22.1, Sentence Transformers 6.2.0, LangChain Core 0.3.86, LangChain Chroma 0.2.6, ChromaDB 1.5.9, and pypdf 6.10.0. The final notebook removes unused torchvision because Colab supplied a version incompatible with the pinned Torch installation. The runtime restart reloads the installed packages before the pipeline starts.

III. EXPERIMENT AND EVALUATION METHODS

A. Chunking and Hardware Experiments

We retained the WW1–WW2 experiments [7] as the configuration-selection evidence. Table I lists five configurations. Chunk size and overlap are measured in characters. A, B, and C vary both size and overlap, so their results do not isolate chunk size alone. B versus D changes overlap, and B versus E changes the number of retrieved passages.

TABLE I
TESTED CHUNKING AND RETRIEVAL CONFIGURATIONS

Run	Chunk	Overlap	Top-k	Devices
A	400	50	3	CPU and GPU
B	800	100	3	CPU and GPU
C	1200	150	3	CPU and GPU
D	800	200	3	GPU
E	800	100	5	GPU

Each configuration/device pair used one unscored Q01 warm-up, followed by three measured trials over the same 12 questions. We rebuilt the index for each trial. The original-prompt experiments produced 288 measured responses. Revised-prompt and prepared-source GPU D follow-ups added 36 each, giving 360 earlier measured responses. Repetitions provide timing observations; they do not create additional independent questions.

The CPU runtime reported an AMD EPYC 7B12 host. The GPU runtime reported a Tesla T4 with an Intel Xeon host at 2.00 GHz. We checked actual model placement on cpu or cuda:0. These were different Colab hosts, so reported speedups compare those observed environments. We did not measure peak memory, cost, concurrency, or cold-start capacity.

Total response time is the sum of retrieval, prompt preparation, model generation, and decoding. We synchronized GPU operations around the timed model call. Source loading, model loading, indexing, scoring, and export are outside that response time. Indexing includes chunk creation, embedding, and vector-index construction. Warm-ups and incomplete attempts are excluded from measured means.

B. Prompt and Source Followups

The revised prompt asks the model to match facts to the exact program and course code, avoid borrowing details from adjacent courses, and answer comparison items separately. It also asks the model to name a specific missing detail when the context is insufficient. We kept D at 800/200/3 and used the original PDF extraction for this prompt follow-up.

A separate follow-up used 1,094 labelled source records with that same revised prompt and D settings. It reorganized curriculum rows, course descriptions, and semester summaries before recursive splitting. This was a combined source-preparation strategy. We did not attribute its result to one formatting operation, and we retained both its gains and regressions.

C. Final Question Set

The final-project instructions require at least 10 questions and both answerable and unanswerable cases [8]. We kept the original 12 answerable questions and added four unanswerable questions, giving 16 in total. The original set has four easy, four average, and four difficult questions. Its reference key defines 32 documentary facts and two answer qualifications. These development questions were already inspected during the earlier experiments.

Q13 asks for BSIT internship hours, Q14 for the required Excel version, Q15 for exact tuition and mandatory fees, and Q16 for the days and times of an R course. Those requested details are absent from the approved source. We fixed their references and missing-information requirements before collecting the responses. We kept the selected model, revised prompt, and original PDF pipeline unchanged for this final evaluation.

On November 10, 2026, the selected T4 GPU pipeline generated one fresh response to each question after one excluded warm-up. It built the index once for this quality evaluation. This run adds 16 measured responses and does not replace the earlier three-trial benchmark. Its source and question hashes matched the package, all 16 questions completed, and all 48 supplied passages matched their saved PDF text positions.

D. Scoring and Source Review

We scored correctness and source support separately from zero to two. Correctness checks emitted factual claims against the reference and source. Source support checks those claims against the passages actually supplied to the model. A correct fact can still lack retrieved support. Completeness is the correctly answered required-element count divided by the fixed count for the question. A prerequisite list counts as one grouped element and must contain every required code.

For answerable questions, full success requires correctness 2, support 2, and completeness 1. Retrieval precision is the share of returned passages that usefully support a requested part under the fixed relevance rules. Documentary coverage measures the required facts fully established by their union. A partial list or a shared course keyword does not establish complete evidence coverage.

For each unanswerable question, U1 requires identifying the specific missing detail, and U2 requires avoiding an invented or borrowed answer. Successful handling requires both. We assess any added claims for support as well. Related passages can be topically relevant without answering the question, so ordinary answer-evidence coverage is not applicable to these four questions. We do not divide by zero or count topical similarity as proof of an absent fact.

Our final scores come from AI-assisted source review using the fixed rubric, actual responses, prompts, and retrieved passages. The original 12 final responses and their ordered context exactly matched the previously source-audited revised-prompt run. Their rubric decisions therefore stayed the same. Earlier group verification and later source auditing are retained in the review history, but the final review does not claim a new independent group verification or measured inter-rater agreement.

IV. RESULTS AND DISCUSSION

The values below come from actual notebook execution. The results ZIP preserves the notebook-generated records, while the Excel workbook and CSV organize the same results for review. We report earlier benchmark means separately from the final 16-question run.

A. CPU and GPU Response Times

TABLE II
MATCHED ORIGINAL-PROMPT RESPONSE TIMES IN SECONDS

Run	CPU	GPU	Speedup
A	44.170	3.381	13.063
B	57.849	4.058	14.254
C	75.075	4.890	15.352

The matched response speedups ranged from 13.063 to 15.352 times. Generation accounted for most of the change. For C, it averaged 75.039 seconds on CPU and 4.873 seconds on GPU, while total response averaged 75.075 and 4.890 seconds. These speedups apply to answering questions under the measured conditions. They do not describe model downloading or the whole notebook.

Indexing also improved. B averaged 14.026 seconds on CPU and 0.855 seconds on GPU. We kept indexing separate because an interactive chatbot can reuse its stored index. Rebuilding an index for every experimental trial provides indexing observations, but adding that entire cost to every deployed response would describe a different workload.

B. Configuration Tradeoffs

TABLE III
ORIGINAL-PROMPT GPU TIMING AND SOURCE-AUDITED QUALITY

Run	Response s	Complete %	P@k %	Coverage %	Full /12
A	3.381	47.9	25.0	35.4	3
B	4.058	62.5	50.0	61.8	3
C	4.890	60.4	44.4	68.1	4
D	4.319	64.6	50.0	63.2	5
E	5.590	54.2	33.3	70.1	4

A was fastest on GPU at 3.381 seconds, but only three questions met every success criterion. D reached five and had higher completeness than A or C. E retrieved five passages and reached 70.1% documentary coverage, but only four answers were fully successful. Its response mean rose to 5.590 seconds. More context did not consistently help the model select and interpret the right course information.

The source audit also changed one earlier C rating. Its ECE C405 answer gave the correct lecture value but cited Source 2, which described other courses; the needed description was in Source 3. Support fell from two to one, reducing C from five to four fully successful questions. This correction did not change the generated answer or measured time.

C. Selected Prompt and Source Representation

TABLE IV
EARLIER GPU D FOLLOWUPS ON THE ORIGINAL 12 QUESTIONS

Variant	Response s	Corr. /2	Supp. /2	Compl. %	Full /12
Original	4.319	1.33	1.25	64.6	5
Revised prompt	3.743	1.50	1.33	70.8	6
Prepared source	4.179	1.33	1.25	60.4	5

The revised prompt added one fully successful answer and raised completeness to 70.8%. Precision and coverage stayed at 50.0% and 63.2%, so it improved use of the retrieved context without fixing every retrieval gap. Its mean output length was 71.25 tokens versus 83.67 for original D. Shorter generation is consistent with its lower measured response time, but separate Colab runs do not isolate the prompt as the sole cause.

The prepared source returned five fully successful answers, with 60.4% completeness and 36.1% retrieval precision. It helped some Game Programming evidence but introduced other regressions. We selected revised-prompt GPU D with the original PDF as our development baseline because it gave the best observed balance among these follow-ups. We did not select a later version merely because its source looked cleaner.

The supplementary Qwen3-8B NF4 GPU response means were 8.570, 11.320, 12.071, 9.864, and 14.473 seconds for A–E. Its interrupted CPU pilots yielded no completed CPU mean or valid CPU/GPU speedup. These records explain the feasibility decision to use Qwen2.5. They do not establish an answer-quality advantage for either model under a matched model comparison.

D. Final Answer Quality

TABLE V
FINAL 16-QUESTION OUTCOMES UNDER CLASS-SPECIFIC CRITERIA

Question class	N	Successful	Rate %	Response s
Answerable	12	6	50.0	3.588
Unanswerable	4	1	25.0	3.558
Overall	16	7	43.75	3.580

The selected pipeline fully answered 6/12 answerable questions and correctly handled 1/4 unanswerable questions. Overall success was 7/16, or 43.75%, using the two stated class-specific criteria. This is a question-level success rate, not a claim that 43.75% of all emitted factual claims were correct. The 12 answerable questions averaged 1.500 correctness, 1.333 support, and 70.8% completeness.

TABLE VI
FINAL RESPONSE SCORES

Question	Correct /2	Support /2	Complete %	Success
Q01	2	2	100	Yes
Q02	1	1	25	No
Q03	2	0	100	No
Q04	2	2	100	Yes
Q05	1	1	50	No
Q06	2	2	100	Yes
Q07	2	2	100	Yes
Q08	1	1	25	No
Q09	2	2	100	Yes
Q10	0	0	0	No
Q11	2	2	100	Yes
Q12	1	1	50	No
Q13	0	0	0	No
Q14	0	0	0	No
Q15	2	2	100	Yes
Q16	0	0	0	No

Q13 showed a unit-category error. The model called the listed six internship credit units six training hours. Q14 transferred Visual Basic for Excel from a neighboring description and presented it as the required Excel version. Q16 invented Monday and Wednesday classes from 10:00 to 11:15. None of those requested details is established by the supplied source.

Q15 correctly said that the available context could not establish exact tuition and mandatory fees. It gave no peso amount. The curriculum total mentioned on filtered PDF page 12 is not a fee assessment, and the answer did not use it to calculate a price. This was the only successful missing-information response in the final set. The prompt already asked for acknowledgement of missing details, so including that instruction did not ensure compliance.

The original questions showed other limits. Q03 gave the correct ECE C405 lecture value, but its retrieved curriculum only established a unit value without the lecture label. It received correctness 2 and support 0. Q05 placed the DAT internship in the first semester instead of the second. Q08 omitted required prerequisites and a requested course. Q10 used whole-program category totals in place of fourth-year semester totals, and Q12 attached a prerequisite footer to the wrong course.

E. Final Timing and Setup

Mean final response time was 3.580 seconds. Retrieval averaged 0.012370 seconds, preparation 0.004034, generation 3.563715, and decoding 0.000262. The sum agrees with the saved response total. All 16 responses finished below the output-token cap. The final 12 answerable responses averaged 3.588 seconds; their mean is distinct from the earlier revised-prompt three-trial mean of 3.743 seconds.

The final run recorded 3.012 seconds for source loading, 22.427 for embedding-model loading, 29.949 for language-model loading, and 1.387 for the single index build. The excluded warm-up response took 2.403 seconds. These are individual setup observations, not repeated setup means. A small response average therefore does not establish equally fast startup.

F. Hosted Functional Checks

On November 10, 2026, we checked three question types in the hosted app: BSCpE program focus, ITC C402 Internship prerequisites and credit units, and an exact first-year BSIT tuition amount. The BSCpE answer matched filtered PDF page 3. The internship answer gave six credit units but omitted ITC C403 from the two prerequisites listed on page 49. Retrieval supplied both codes, so this was an incomplete generated answer. The corrected tuition request acknowledged that the context did not supply a fee and gave no amount.

We retained eight downloaded response records across the initial checks, interface updates, and final confirmation. One accidental tuition input included a list number and quotation marks. That response invented a cost of three pesos per unit from academic-unit entries. Removing the list number produced a response that acknowledged the missing fee. The inputs and passage order differed, so we treat this as an observed input-formatting sensitivity, not proof that the visual update caused or fixed an error. We kept the unsuccessful record as additional evidence.

For the final interface confirmation, the BSCpE request took 1.257 seconds after setup and 6.032 seconds of server processing. The latest internship check recorded 0.627 and 0.833 seconds, while the corrected tuition check recorded 1.885 and 6.170 seconds. Server processing includes GPU allocation and index opening or building, but excludes Gradio queue and network time. These are individual functional checks collected at different points, not repeated benchmark means. They do not replace the 16-question evaluation or establish a speedup over the T4 run. The final interface returned its answer and a complete downloadable record.

V. LIMITATIONS AND FURTHER WORK

This is a small development study from one handbook edition. The final test has 16 questions, and the original 12 were already used during configuration and prompt development. The four new questions extend the boundary test, but the entire set cannot be called held out. Three repeated timing trials do not increase the number of distinct questions. We report descriptive means without a significance claim.

The implemented study also differs from the broader proposal. We used the 2026–2027 source instead of its initial 2025–2026 edition and evaluated 16 questions instead of the planned approximately 40. We did not complete the planned comparison against the same model without retrieval, a prospective-student needs study, or independent reviewer agreement analysis. Our results therefore describe this RAG implementation and its tested settings; they do not measure the causal benefit of retrieval or improved student decisions.

Flattened two-column tables, page continuations, and similar course codes remain important failure sources. MiniLM normally truncates inputs beyond 256 word pieces [5], so larger character chunks may not be fully represented in an embedding. The system also lacks a validated answerability detector. The current results do not support using it as an authoritative advising or scheduling service. The hosted checks also cover only selected individual requests; we did not measure usability, mobile-device performance, service availability, or multiuser load.

Further work should use separate development and evaluation questions. We would test course-aware retrieval, stronger program metadata, and reranking as identified new experiments. Missing-information handling needs both testing and a clear operational boundary. If we revise the prompt after seeing Q13–Q16, we must evaluate that revision on additional questions rather than present these same four as untouched evidence.

VI. REPRODUCIBILITY AND RESEARCH INTEGRITY

The final notebook preserves the supplied template flow in Sections 1–9. Section 10 recalculates earlier experimental summaries from their saved execution records. Section 11 collects and displays the final 16 responses. Section 12 displays the reviewed scores and findings. The appended review outputs were calculated from the exported Colab records after execution; no model generation was repeated during scoring.

The supporting package contains the original and filtered source documents, fixed question files, reference excerpts, prepared-source records and scripts, raw trial logs, complete prompts, and ranked passages. It also preserves earlier and supplementary experiments. The workbook includes the final results, required answer elements, exact source locations, earlier benchmark tables, and setup records. The CSV contains each question, its reference, actual answer, ratings, reasons, and stage timings. These results originate in the notebook; the archive transports the saved evidence.

Each final judgment is bound to the exact question, answer, full prompt, and ordered retrieved passages. The notebook refuses to reuse a judgment if that content changes. Model weights and working indexes are rebuilt. Pinned revisions and source hashes make the procedure traceable, but do not guarantee identical timing or bitwise-identical outputs across Colab sessions. New outputs require a new source review.

The deployment files include the interface, ZeroGPU runtime wrapper, unchanged notebook prompt and answer functions, fixed settings, dependency list, and filtered PDF. The hosted JSON records preserve the submitted questions, full prompts, ordered passages, software versions, hardware, and timings. We retain these separately from the notebook evaluation records so their different execution conditions remain clear.

We disclose AI assistance in the response scoring and report preparation. Saved model outputs and raw timings remain

unchanged, and the review history distinguishes source auditing from group verification. This study does not claim independent blinded scoring.

VII. CONCLUSION

Our RAG chatbot integrated PDF processing, overlapping chunks, MiniLM embeddings, Chroma retrieval, and local Qwen generation. The earlier matched experiments showed substantially faster responses on the tested GPU runtime. Revised-prompt D provided the best observed development balance among the tested follow-ups, while prepared source records did not consistently improve the answers.

The final evaluation showed why speed and answer reliability must be assessed separately. The system achieved six fully supported and complete answers out of 12 answerable questions, but handled only one of four missing-information questions correctly. The browser deployment makes the prototype accessible for individual trials, while its functional checks retain the same course-completeness limitation. We retain it as a development baseline. More reliable course attribution and testing on new questions are needed before broader use.

REFERENCES

[1] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474. [Online]. Available: https://arxiv.org/abs/2005.11401

[2] United Nations, "Goal 4." Accessed: Nov. 10, 2026. [Online]. Available: https://sdgs.un.org/goals/goal4

[3] Jose Rizal University, College General Information, School Year 2026–2027. Institutional handbook, CSE sections. Submitted source: COL-GI-2026-2027_CSE_Only.pdf, 64-page excerpt with original-page map.

[4] Qwen Team, "Qwen2.5-1.5B-Instruct," model card. Accessed: Nov. 10, 2026. [Online]. Available: https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct

[5] Sentence Transformers, "all-MiniLM-L6-v2," model card. Accessed: Nov. 10, 2026. [Online]. Available: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[6] Qwen Team, "Qwen3-8B," model card. Accessed: Nov. 10, 2026. [Online]. Available: https://huggingface.co/Qwen/Qwen3-8B

[7] Jose Rizal University, "ITC-C508 Lab Exercise WW1 and WW2 (Optimization and GPU Testing): Student Experiment Guide," course handout, n.d.

[8] Jose Rizal University, "ITC 508 (Deep Learning and NLP): Final Project: RAG Chatbot Development and Evaluation," final-project deliverables handout, n.d.

[9] DecisionThree, "JRU CSE Assistant," Hugging Face Space, deployed and tested Nov. 10, 2026. [Online]. Available: https://huggingface.co/spaces/loid00/decisionthree-jru-cse-chatbot

[10] Hugging Face, "Spaces ZeroGPU: Dynamic GPU Allocation for Spaces," documentation. Accessed: Nov. 10, 2026. [Online]. Available: https://huggingface.co/docs/hub/spaces-zerogpu